A PROJECT REPORT

on

"MULTIMODAL DEMAND FORECASTING: HYBRID CNN-GRU, NLP AND COMPUTER VISION FUSION WITH RETRIEVAL-BASED EXPLANATION"

Submitted to

KIIT Deemed to be University

In Partial Fulfilment of the Requirement for the Award of

BACHELOR'S DEGREE IN

COMPUTER SCIENCE & SYSTEM ENGINEERING

BY

SOHAM CHATTERJEE	**2328128**
AVRRODEEP BANERJEE	**2328158**
DEBARSHI KARMAKAR	**2323164**
SUPRATIK MAITY	**2328226**

UNDER THE GUIDANCE OF

Dr. Bhaswati Sahoo



SCHOOL OF COMPUTER ENGINEERING

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY

BHUBANESWAR, ODISHA - 751024

November 2026

KIIT Deemed to be University
School of Computer Engineering
Bhubaneswar, ODISHA 751024

CERTIFICATE

This is to certify that the project entitled

"MULTIMODAL DEMAND FORECASTING: HYBRID CNN-GRU, NLP AND COMPUTER VISION FUSION WITH RETRIEVAL-BASED EXPLANATION"

submitted by

Soham Chatterjee	**2328128**
Avrrodeep Banerjee	**2328158**
Debarshi Karmakar	**2328164**
Supratik Maity	**2328226**

is a record of bona fide work carried out by them, in partial fulfilment of the requirement for the award of the degree of Bachelor of Engineering/Technology in Computer Science & System Engineering at KIIT Deemed to be University, Bhubaneswar. This work was done during the year 2026-2027, under our guidance.

Date: 10 / 11 / 2026

Dr. Bhaswati Sahoo
Project Guide

ACKNOWLEDGEMENTS

We are profoundly grateful to Dr. Bhaswati Sahoo of the School of Computer Engineering, KIIT Deemed to be University, for expert guidance and continuous encouragement throughout the project, from the choice of topic to the final report. We also thank the faculty and staff of the School for their support, and our families and friends for their patience.

Soham Chatterjee
Avrrodeep Banerjee
Debarshi Karmakar
Supratik Maity

ABSTRACT

Forecasting of demand for e-commerce is typically done on the basis of sales data alone, but customer reviews and photos have some early signals about the changes in demand. This work presents a hybrid multimodal forecasting framework to forecast the next 14 days of demand for every product using three types of data input – sales history, processed using GRU recurrent neural network and 1D convolutional neural network; customer review text data, processed using TF-IDF, LSA, and small recurrent encoder; and photos of products processed using convolutional neural network. An explanation module retrieves relevant reviews.

Two methods of fusing the inputs are developed and compared using the same dataset: staged fusion, where the separate encoder models are fed into a LightGBM classifier, and joint fusion, where a single model is trained end to end with a gate that randomly masks entire modalities. Both methods are evaluated chronologically on a 120-day hold out with gaps that prevent leakage, in comparison to four baseline methods, on three different seeds, with an ablation, robustness check and a control dataset where reviews and images contain no information. The dataset is synthetic by default; however, there is an option to upload the user's own sales, reviews and images, any of which except sales may be incomplete.

On the synthetic data, the review text reduces the WAPE error by approximately 0.02 WAPE; the best-performing model achieves 0.179 in WAPE compared to 0.234 for the seasonal-naive rule. Staged and joint fusion approaches seem equally accurate; the staged approach is barely affected by the noise, whereas joint performs worse by 0.01 WAPE due to fitting noise in the control experiment. They degrade gracefully when one of the input modalities gets dropped for forecast. The system is provided as a ready-for-use Streamlit application with 31 test cases built-in.

Key words: demand forecasting, multimodal learning, GRU, CNN, review text, LightGBM, fusion, retrieval-augmented explanation, missing modalities

CONTENTS

Page No.

1 Introduction....1
2 Basic Concepts / Literature Review....1
2.1 Forecasting, Baselines and Metrics....1
2.2 Neural Networks, Boosting and Text Features....1
2.3 Multimodal Fusion, Explanation and Leakage....2
2.4 Related Work....2
3 Problem Statement / Requirement Specifications....2
3.1 Project Planning....2
3.2 Project Analysis (SRS)....3
3.3 System Design....3
3.3.1 Design Constraints....3
3.3.2 System Architecture / Block Diagram....3
4 Implementation....4
4.1 Methodology / Proposal....4
4.1.1 Synthetic Data....4
4.1.2 Ingestion, Windows and Chronological Split....5
4.1.3 Encoders and the Two Fusion Paths....5
4.1.4 Evaluation, Explanation and Application....7
4.2 Testing / Verification Plan....7
4.3 Result Analysis / Screenshots....7
4.3.1 Sample Run on the Team's Own Machine....9
4.4 Quality Assurance....13
5 Standards Adopted....13
5.1 Design Standards....13
5.2 Coding Standards....13
5.3 Testing Standards....13
5.4 Cost Analysis....13
6 Conclusion and Future Scope....14
6.1 Conclusion....14
6.2 Future Scope....14
References....14

List of Figures

1.1 Overview: three optional inputs, three encoders, fusion, forecast and explanation....1
3.1 Layered architecture of the system....4
4.1 Synthetic data: sales follow hidden quality about 6 days later; image examples at four quality levels....5
4.2 Chronological split with gaps and the data used by each fusion path....5
4.3 The three encoders and the shared output head....6
4.4 The two fusion strategies compared....6
4.5 WAPE of every approach on the signal data (3 seeds; whiskers: standard deviation)....8
4.6 Control experiment and robustness: (a) key approaches on signal and control data; (b) inputs removed at forecast time..8
4.7 Comparison tab: verdict, error chart and table....9
4.8 Forecast and explanation tab: 14-day forecast, explanation and retrieved reviews....9
4.9 Sample run, control data (9 products, 1000 days): error on the hold-out period for all approaches, with seed whiskers..10
4.10 Sample run: error by forecast distance with all approaches selected....10
4.11 Sample run: error by forecast distance for the two full-input models, Path A (staged) and Path B (joint)....11
4.12 Sample run: robustness table with inputs hidden at forecast time....11
4.13 Sample run: 14-day forecast for kettle_1 with Path B (joint network)....12
4.14 Sample run: choosing the product from the drop-down list....12
4.15 Sample run: 14-day forecast for backpack_3 with Path A (staged encoders + LightGBM)....12

List of Tables

2.1 This project compared with typical existing work....2
3.1 Project plan....2
3.2 Functional requirements....3
3.3 Modules....4
4.1 Inputs given to the models....5
4.2 Configuration....6
4.3 Test cases....7
4.4 Signal data, 3 seeds (mean ± standard deviation)....7
4.5 Numbers shown in the sample run (control data, 9 products, 1000 days, hold-out WAPE)....13

Chapter 1 Introduction

Demand forecasting tries to determine how many units of each product the customers would purchase in the coming days. It helps retailers determine how much inventory to purchase and how many employees to employ. An overestimate leads to loss of sales while an underestimate results in tying down money in the unsold inventory. However, most of the forecasting techniques rely only on the sales data while in an online store, there is past data available about changes in the form of a series of complaints about a faulty component which is normally followed by lower sales and quality control photos showing damage.

In this research, we develop an integrated multimodal system for demand forecasting that takes into account three types of information – the historical data on sales (GRU recurrent neural network with 1D-CNN layer), customer reviews (natural language processing) and pictures of products (CNN for computer vision), providing the prediction of the next 14 days demand for each product. There is a retrieval-based explanation model that highlights the relevant reviews with regard to the predicted forecasts (a simple version of retrieval-augmented generation, RAG). The main problem to be solved is the way how different modalities can be combined, that is why we propose two solutions that will be evaluated with the same dataset: Path A (staged fusion) and Path B (joint fusion).

The project fills five gaps in practice: (i) public projects rely on sales alone; (ii) they employ random splits or features with information leakage to the future; (iii) data may be incomplete, but software always breaks down on missing reviews and images; (iv) different fusion techniques are never compared in equal conditions with repetitions; and (v) one single figure does not give the planner anything to verify. The data is artificial, in order to make sure all modalities are complete and their relationship with demand is clear, and there is also a control setting where reviews and images have no information content at all. The user can submit their own files, and any input but sales can be missing. Chapter 2 discusses the concepts and related literature, Chapter 3 – the requirements and architecture, Chapter 4 – the implementation, tests and results, Chapter 5 – the standards and cost, and Chapter 6 – the conclusions.

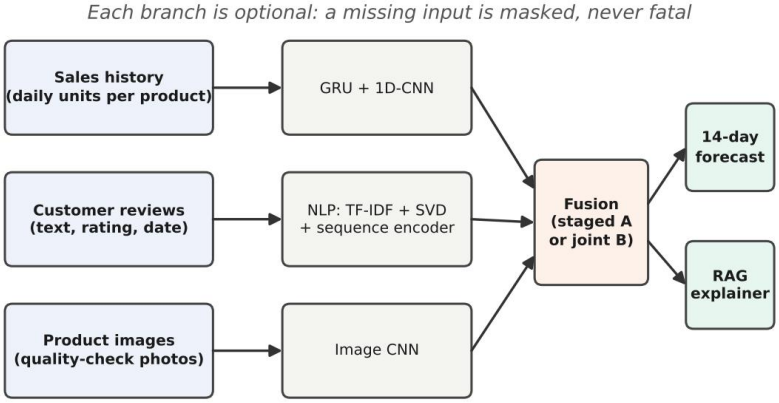


Figure 1.1: Overview: three optional inputs, three encoders, fusion, forecast and explanation

Chapter 2 Basic Concepts / Literature Review

2.1 Forecasting, Baselines and Metrics

In demand series, there are elements of level, trend, seasonality (weekly and annually), and noise [1]. In this case, we make a prediction for all 14 days at once instead of making a sequential prediction in order to take into account that in the future we will not know how many reviews and pictures a product will have. We compare models using simple benchmarks: average value, average of last 7 days, and seasonal benchmark (the same day of previous week). The primary criterion is WAPE=$\Sigma|y-\hat{y}|/\Sigma|y|$—the total absolute error in relation to total sales (0.18 means 18% error rate); the secondary is MASE [2], where we divide a product's error by the weekly naïve error of this product**.**

2.2 Neural Networks, Boosting and Text Features

A GRU [4], a less complex variant of the LSTM [3], processes a sequence incrementally using gated memory, while a 1D-CNN applies filters over time to discover local patterns like a weekend peak. Similarly, a 2D-CNN [5] processes images. LightGBM [6] builds a number of small decision trees, one correcting the

previous one, and deals with missing are intrinsic to Path A and are used for the absence of modalities. Reviews are encoded via TF-IDF [7] weighting more distinctive words, and then condensed using Latent Semantic Analysis [8] (truncated SVD) into 16 numbers per product per day along with mean rating, logarithm of review count, and had-reviews indicator. Fitting is done on pre-hold-out reviews only.

2.3 Multimodal Fusion, Explanation and Leakage

The multimodal approaches [9] vary in terms of the stage at which the data types are combined. In this work, two types of intermediate fusion will be created: the staged fusion, which is a stacking approach [10] whereby each encoder is trained independently, frozen and another model trains on the embeddings (it is easy to inspect but each stage has access to partial data); and joint fusion whereby the encoders and head share the same loss function (uses all data but might fit noise from uninformative inputs). For missing inputs, the modality dropout [11] approach that randomly hides whole modalities during training will be used. The RAG [12] approach involves retrieving relevant documents and then generating text; in this case, the retrieval is genuine while the generation is through templates and thus the explanation contains only computed numbers. The data leakage [13] is the use of future information; the defense against time series data is the chronological validation [1], [14].

2.4 Related Work

Gradient boosted trees trained on lagged and calendar data were the winners of the M5 retail competition [15]. The neural models used for forecasting are DeepAR [16] and the Temporal Fusion Transformer [17] that work on numerical covariates but do not consider text and images. Multimodal sales forecasting literature has concentrated on novel items without past records [18].

Table 2.1: This project compared with typical existing work

Aspect	Typical public projects	Published research [15]-[18]	This project
Inputs	Sales, calendar	Numeric covariates; images/text for new items	Sales, reviews, images
Missing inputs	Not handled	Rarely discussed	Masked, reported, still runs
Fusion compared	None	One design	Staged and joint, same data
Evaluation	Often random split, one run	Chronological	Chronological, gaps, 3 seeds, control
Explanation	None	Variable importance	Retrieved reviews, sensitivity

Chapter 3 Problem Statement / Requirement Specifications

Problem statement. Demand forecasting usually involves sales only even though there is early information about changes available in reviews and pictures, while current approaches do not integrate both types of information or handle situations where some of it is unavailable. Thus, the aim is to create, develop, and test a system that would forecast demand 14 days ahead for each product based on sales, text of reviews, and images; compare two fusion approaches in ideal conditions; operate with any optional input missing; and provide justification for each prediction. Moreover, the system needs to be demonstrable using non-private data and should prove that it does not produce any gains in case of uninformative reviews and images.

3.1 Project Planning

This was divided into eight phases, taking approximately two weeks altogether (Table 3.1), and at the end of each phase there is a verifiable result.

Table 3.1: Project plan

No.	Phase	Outcome	Time
1	Study and scoping	Existing projects reviewed; scope fixed (three inputs, two fusion paths, synthetic data, upload)	Days 1-2
2-3	Synthetic data, ingestion	Generator with signal and control modes; column detection, imputation, status and fallbacks	Days 3-4
4-5	Windows and models	Leak-free windows; GRU/CNN encoders, joint network, staged stacker	Days 4-8
6-7	Benchmark, explainer, app	Metrics, robustness, verdict; ridge retrieval; Streamlit application; 31 tests	Days 8-13

No.	Phase	Outcome	Time
8	Documentation	README, sample data, report, paper	Day 14

3.2 Project Analysis (SRS)

Requirements are organized according to the format used in IEEE 830 and ISO/IEC/IEEE 29148 [19], [20]. Scope: the system provides the software hff that makes predictions for 12 items up to 14 days in advance, compares and evaluates two fusion methods and provides predictions; it does not include automatic ordering of stock and database connection with stores. Users: an analyst that interacts with the web interface and developer that runs command line benchmark. Local execution on Windows, Linux, macOS with Python 3.10+.

Table 3.2: Functional requirements

ID	Requirement	Module
FR1-3	Accept a sales CSV (date, demand and product columns detected from common names; missing date or product column allowed), an optional reviews CSV (date, text, optional rating/product) and an optional images ZIP (manifest or dates in file names; unreadable images skipped).	ingest
FR4	Generate synthetic sales, reviews and images in a signal mode and a control mode.	synth
FR5-6	Report per input whether it is usable, its size, coverage and reason if not; run with any subset of inputs; if only reviews exist, use review count as a flagged demand proxy.	ingest, app
FR7-8	Train baselines, LightGBM, Path A and Path B for every supported input combination on one chronological hold-out; report WAPE, MASE, error by forecast day and spread across seeds.	fusion, benchmark
FR9-10	Test models with text, images or both hidden; give a plain-language verdict and decline to name a winner within seed noise.	benchmark
FR11-12	Forecast 14 days per product (Path A or B) with and without each modality; explain with change from recent level, sensitivity, influential words and retrieved reviews.	forecast, explain
FR13-14	Offer CSV downloads and a command-line benchmark; show a readable message, not a traceback, for unusable or too-short data.	app, scripts

Non-functional requirements:correctness (no use of future information in the inputs; test verified); reproducibility (seeded, results in the form of mean ± standard deviation); performance (full benchmarking takes approximately 5–9 minutes using a two-core CPU); robustness (optional inputs can be missing and/or malformed); portability (all packages are pure-Python packages); privacy (local computing only, 5MB for each image, up to 20,000 images). Ambiguities in analysis:

Demand without sales: the number of review counts per day is used as a proxy, the run is marked and the comment says "review volume".

- Absent days: in transaction records there are no transactions in absent days; in daily panels there are up to 7 days interpolated, over 7 imputed to the median value of the product and there is an indicator of an imputed day sent to the algorithm.
- Matching products: the key is generated from columns with names such as store, item, product, category, sku or series; with one sales product, the unmatched reviews and pictures are matched to this product; otherwise, the input is turned off by a specific reason.
- Not enough data: less than 50 reviews, less than 2 % product-days coverage, less than three images, less than 60 observed days (a product) or less than 150 training windows turn off the input or terminate the run by the required number of days. Pictures older than 21 days are absent days.
- True differences: the gap below two times the sum of seed standard deviation (0.01 WAPE per seed) is labeled as "no clear difference".

3.3 System Design

3.3.1 Design Constraints

This system is implemented in Python. Experiments were conducted under Linux (Python 3.13, pandas 3.0, scikit-learn 1.9, LightGBM 4.7, Keras 3.15 on TensorFlow, Streamlit 1.65); the tests were rerun under Windows (using the version pip installs for Python 3.10 – pandas 2.3.3, NumPy 2.2.6, scikit-learn 1.7.2, Keras 3.12.4, TensorFlow-CPU 2.21.0) and passed in both cases. Experiments were performed using single-machine CPU-only configuration with two cores and roughly 7 GB of RAM; overall benchmark takes 5.2 minutes for one seed and 8.6 minutes for three. Images are resized to 32 × 32 pixels, text uses TF-IDF and

SVD without pre-trained language model, at most 12 number of products processed per batch, and TensorFlow/Keras 3 is employed since PyTorch installation was impossible in the development environment.

3.3.2 System Architecture / Block Diagram

Figure 3.1 consists of five layers. The ingestion layer converts any input into one package, namely, a sales matrix of products by days, an observed/imputed mask, a review matrix, images for each product and a status row for each input. The features are generated solely on the basis of the package, hence the models will not see the files.

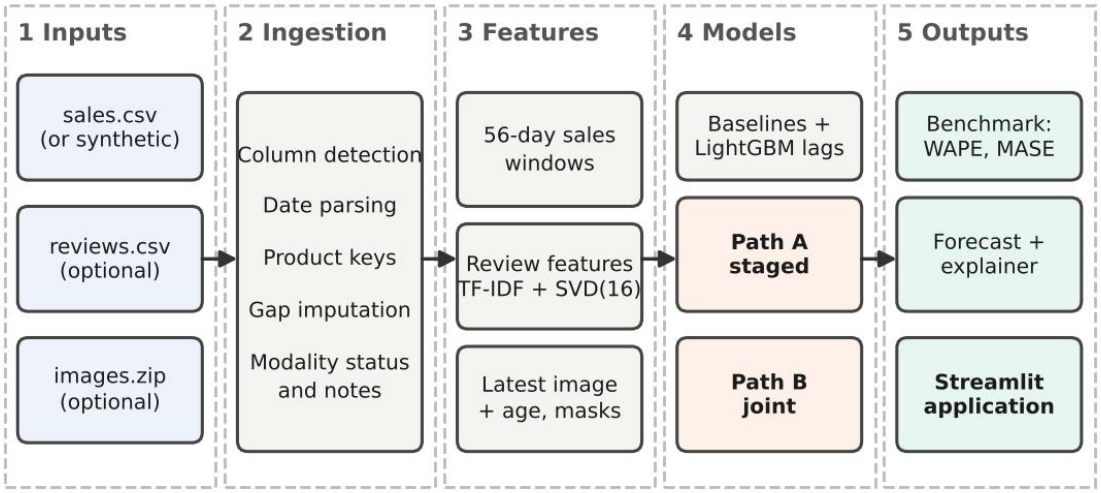


Figure 3.1: Layered architecture of the system

Table 3.3: Modules

Module	Responsibility
config.py, synth.py	Frozen configuration (shrinks for short series or states the days needed); synthetic generator
ingest.py	Column aliases, date parsing, product keys, imputation, reviews-only proxy, image ZIP reader, modality status
windows.py	Chronological split, calendar features, daily review features, forecast-origin windows, modality hiding
nets.py, fusion.py	GRU/CNN/image branches, modality gate, joint model; LightGBM baseline, Path A stacker, Path B predictor
benchmark.py	WAPE, MASE, by-horizon error, robustness scenarios, verdict
forecast.py, explain.py	Final 14-day forecast with hidden modalities; ridge impact model, retrieval, template explanation
app.py, scripts/, tests/	Streamlit interface; command-line benchmark; 31 automated tests

Chapter 4 Implementation

4.1 Methodology / Proposal

4.1.1 Synthetic Data

Each product has an underlying quality process q(t) in [0, 1] that consists of a slowly wandering time series x(t) = 0.97 x(t − 1) + some noise with a mean value of about 0.75 and occasional defect periods (14 to 35 days long, with depth 0.25 to 0.5). The expected number of sales on day t is λ(t) = base × trend × weekday × year × promotion × exp(γ (q(t − 6) − μ)), where base is 30 to 120 units, γ = 1.8 and a 6-day lag, sales are Poisson distributed with mean λ(t). Reviews come in at 0.05 per unit sold 3 days before, using texts formed from template sentences with sentiment following q and a rating of 1 + 4s. A 32 × 32 product image is generated every 7 days with scratchiness increasing with quality (Figure 4.1). In signal mode the review and image quality are the demand quality, thus they precede the sales by about 6 days; in control mode they are independent processes with the same distribution. The data set consists of 6 products × 900

days: 5,400 sales values, 25,353 reviews and 771 images.

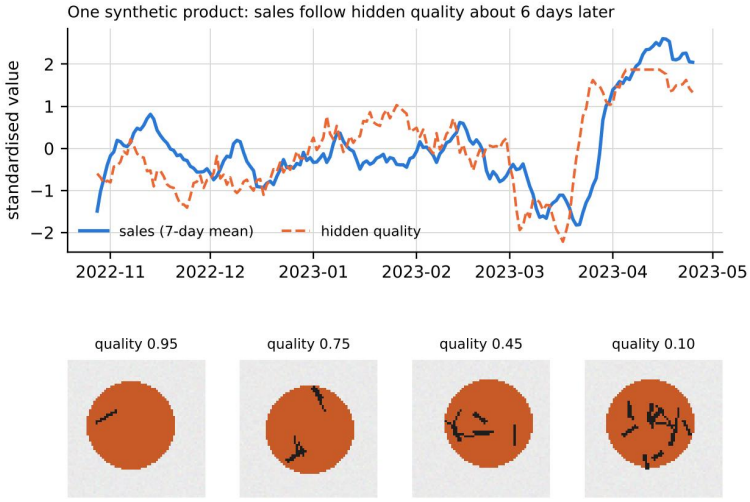


Figure 4.1: Synthetic data: sales follow hidden quality about 6 days later; image examples at four quality levels

4.1.2 Ingestion, Windows and Chronological Split

Ingestion follows the principles of Section 3.2. A forecast origin T is a day when the forecast is originated by the model: the model observes sales of days T−55 to T, review features of days T−13 to T, the newest picture, and forecasts sales days T+1 to T+14. Sales are scaled based on the product's training average; the networks forecast the relative deviation from the recent level (the mean over 7 most recent scaled days), using absolute error loss; the output of the model is level + relative change. The last 120 days constitute the holdout period (642 test origins); a 14-day gap before that guarantees that training targets are not included into the holdout and provides 4,266 training windows; 15 % of training origins after the additional gap are used for early stopping (Figure 4.2).

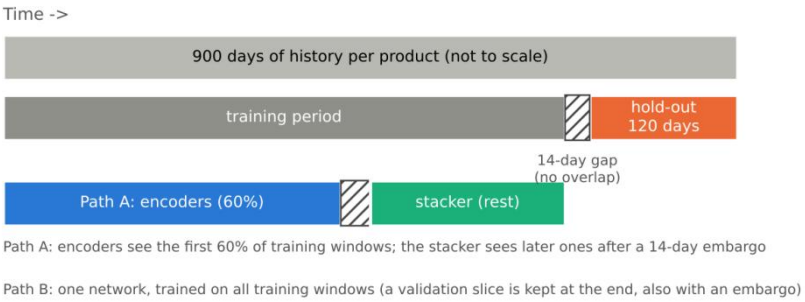


Figure 4.2: Chronological split with gaps and the data used by each fusion path

Table 4.1: Inputs given to the models

Input	Shape	Content
Sales window	56 x 6	Scaled sales, observed flag, sin/cos of weekday and day of year
Review window	14 x 19	16 LSA components, mean rating, log count, had-reviews flag (absent days marked missing)
Image	32x32x3 + 2	Latest image on or before the origin, its age and a presence flag
Hand-built features	10	Last value, lags 7/14/28, means 7/14/28, 7-day standard deviation, trend, observed share (LightGBM)

4.1.3 Encoders and the Two Fusion Paths

The sales encoder passes through a 32-element GRU and a pair of 1D-convolutions (16 filters, kernels of size 7 and 5, global average and maximum pooling) in parallel, producing a 32-element embedding. The text encoder consists of a 1D-convolution followed by a 16-element GRU producing a 16-element output, while the image encoder comprises three 3 × 3 convolutions (8, 16 and 16 elements, respectively) with pooling and produces a 16-element output (Figure 4.3). The common head connects embeddings to a 64-element dense layer after concatenaion with the calendar of the target 14 days to output 14 changes.

Parameters: sales-only 14,510, with text 18,430, with text and image 23,790.

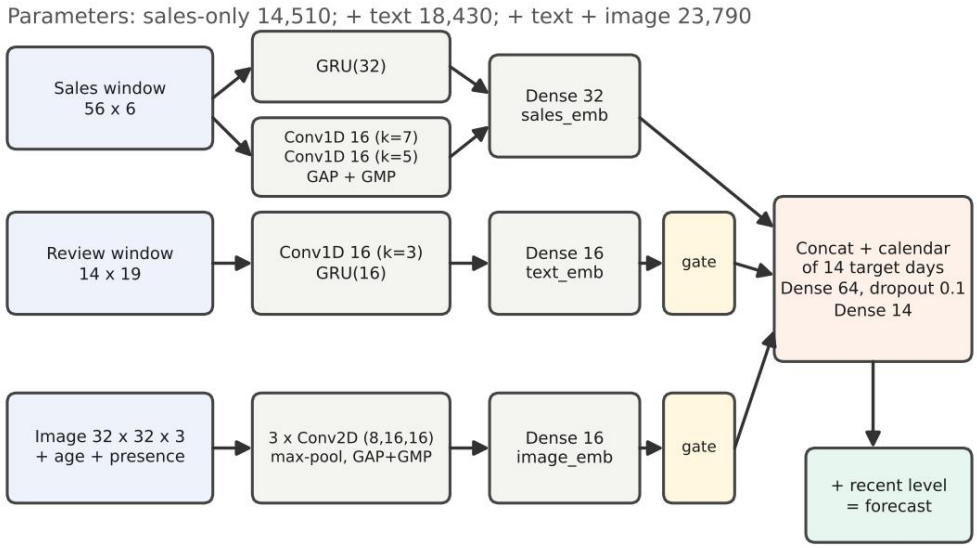


Figure 4.3: The three encoders and the shared output head

Path A (staged). (1) Each encoder is independently trained on the first 60 % of training origins. (2) The encoders are frozen and provide embeddings for the remaining origins, which started more than 14 days after the encoders' origins; missing or unseen modalities are considered missing, and a random 25 % of the samples are considered missing in training. (3) One LightGBM (250 trees, learning rate 0.04, 15 leaves, L1 objective) is trained on one row per sample and forecast day, with the ten features, day number, same weekday last week, four-week weekday average, target-day calendar and embeddings; its output is added to the recent level. Path B (joint). One neural network has all three branches; a modality gate multiplies each text and image embedding by a presence flag, and in training zeros out entire modalities with probability 0.25, so that the network can learn to forecast without them. One loss trains all components on all training origins (Figure 4.4).

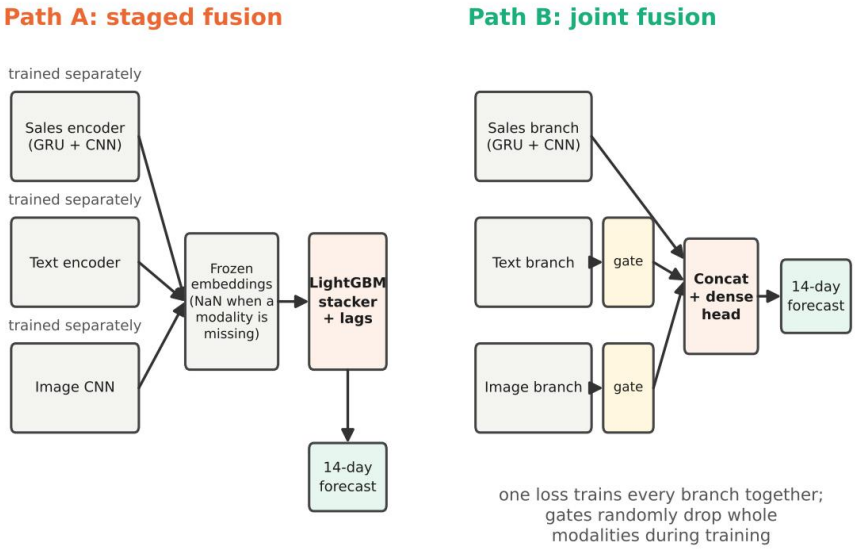


Figure 4.4: The two fusion strategies compared

Table 4.2: Configuration

Setting	Value	Setting	Value
horizon	14 days	epochs / patience	40 / 6
window (sales / text)	56 / 14 days	batch / learning rate	128 / 0.002 (Adam [21])
hold-out	120 days	modality dropout	0.25
image size / max age	32 px / 21 days	validation / encoder share	15 % / 60 %

Setting	Value	Setting	Value
LSA dimension	16	seeds	3 (mean ± sd)

4.1.4 Evaluation, Explanation and Application

Each method is evaluated using the same 642 test origin after product scale multiplication; WAPE and MASE are averaged across all products, origins, and forecast days while WAPE is also recorded for each forecast day. Methods include the three baselines, LightGBM with lags, and paths A and B per each input combination method supported. All methods using all the input are also evaluated without text, image, or both. Verdict names the lowest error method and compares joint and staged fusion, input data against sale data, and neural networks against LightGBM if the difference exceeds twice the combined standard deviation.

The chosen path for the last forecast is used for training on all origins with known targets and making predictions on the last day both with and without all modalities where the sensitivity is defined as the relative difference between the mean forecast and the recent value. The explanation is made using ridge regression (alpha 5) on TF-IDF vectors of pre-hold-out reviews to standardised differences between means of sales from the past 7 days to the next 7, while the extreme coefficients are those words preceding decrease or increase in sales. Three reviews out of the last 14 days that have the lowest (in case of decrease) or highest (in case of increase) are found using a template ending with an indication that this is not a language model output. The Streamlit app consists of a sidebar (data source, sliders, uploads, training, run button) and three tabs: Data check, Comparison and Forecast and explanation (Figures 4.7 and 4.8).

4.2 Testing / Verification Plan

The Verification includes the following layers: unit testing of the generator and ingest functions; leakage property testing (the data following origin is rewritten, while the origin's inputs should not be modified; no scaling when hold-out sales are scaled to 7 times the amount); integration testing using small-scale implementation of both path and performing the benchmark, forecaster and explainer on it; and system testing by using web application driverless. Test functions are represented in table 4.3. They have been tested in two packages environment and passed successfully within 122 s on Linux with the latest versions of packages; 129 s with the current Windows/Python 3.10 package versions). Web application also passed the web server healthcheck.

Table 4.3: Test cases

ID	Test case	Condition	Expected result
T1	Generator	Same seed twice; signal vs control lead correlation; output counts	Identical; > 0.15 vs < 0.1; counts above minima
T2	All inputs / sales only	Sales + reviews + images; sales alone	Usable flags correct; run continues
T3	Partial inputs	Sales + reviews; sales + images; reviews for one product only	Right modality enabled; no crash
T4	Reviews only	No sales file	Review-count proxy, flagged
T5	No data / too short	Nothing provided; 60 days	Readable error with minimum days
T6	Messy columns	"Order Date", "Item", "Units Sold", "Comments", "Stars"; no rating/product; no date	Columns detected; notes shown
T7	Missing days	10 % of rows removed	Gaps filled, flagged, values finite
T8	Bad images	Corrupt file, text file, undated names, only 2 images	Skipped or switched off with reason
T9	Split and leakage	Gap between training and hold-out; overwrite data after origin; scale from training only	No overlap; inputs unchanged; scale equal
T10	Masked inference	Joint model with text, images, both hidden	Finite output; hidden differs from full
T11	Benchmark	Full data; sales-only data	Both paths, baselines, >= 6 robustness rows; no text/image rows when absent
T12	Forecast and explanation	Path A, Path B (full data); sales only	(3, 14), non-negative; <= 3 evidence rows; sales-only wording
T13	Application	Start; synthetic run and forecast; upload sales only; empty upload	No exception; status yes/no/no; readable error

4.3 Result Analysis / Screenshots

Result shown as WAPE on the 120 days holdout (lower is better), averaging over three seed values for the signal dataset (Table 4.4, Figure 4.5). The best approach is staged fusion using all inputs (0.179 ± 0.003), beating seasonal naive by 23.3% (0.234) and about 11% the best approach using only sales input (0.202).

Table 4.4: Signal data, 3 seeds (mean ± standard deviation)

Approach	WAPE	MASE	vs seasonal-naive
Path A staged, sales + text + image	0.179 ± 0.003	0.869	+23.3 %
Path B joint, sales + text	0.181 ± 0.002	0.876	+22.5 %
Path B joint, sales + text + image	0.184 ± 0.005	0.890	+21.3 %
Path A staged, sales + text	0.185 ± 0.003	0.893	+20.8 %
Path A staged, sales + image / sales	0.202 ± 0.001 / 0.002	0.975 / 0.976	+13.8 % / +13.5 %
Path B joint, sales + image / sales	0.204 ± 0.007 / 0.208 ± 0.009	0.981 / 0.997	+12.8 % / +11.3 %
Last-7-day mean / LightGBM lags	0.209 / 0.212 ± 0.003	1.016 / 1.003	+10.8 % / +9.3 %
Seasonal-naive / Mean baseline	0.234 / 0.284	1.134 / 1.344	0 % / -21.2 %

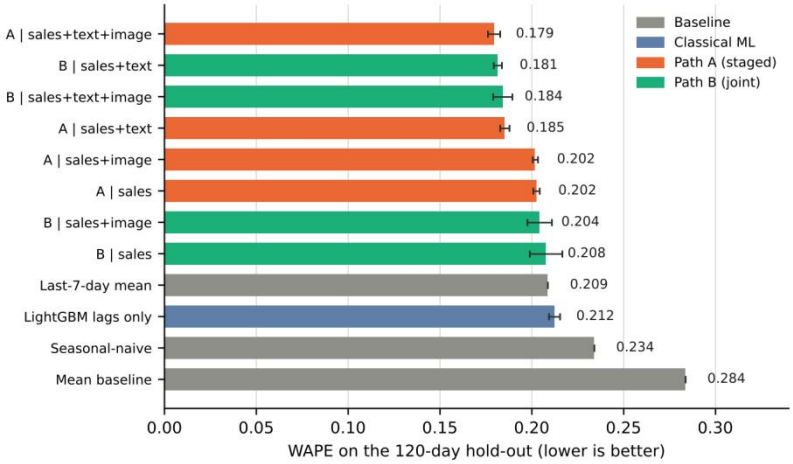


Figure 4.5: WAPE of every approach on the signal data (3 seeds; whiskers: standard deviation)

- **Text helps clearly:** •WAPE reduces from 0.208 to 0.181 (Path B) and 0.202 to 0.185 (Path A), many times the seed dispersal rate.
- **Images help little:**individually they do not contribute anything; together with text they decrease Path A (0.185 to 0.179) but not Path B (0.181 to 0.184). The CNN for images is small and trained from scratch.
- **Joint versus staged: no clear winner•** (.184 versus .179 when all inputs are used; .181 versus .185 when just text is used); these differences fall within the range of seed noise.

Control experiment (Figure 4.6a, single seed). In case of uninformative reviews and images, there is nothing that can help: the staged model remains the same with text (0.236) and is even worse with all inputs (0.239), while the joint network becomes worse (0.247 sales only, 0.257 all inputs), and by removing text reduces its error to 0.239, implying that it had learned to use noise. Robustness (Figure 4.6b). Removal of images makes almost no difference, while removal of text increases the error to 0.198 (Path A) and 0.202 (Path B), and removal of both to 0.202 and 0.203 – not far from sales-only models trained from scratch (0.202 and 0.208) and better than LightGBM on lags (0.212), so both designs gracefully degrade to sales-only accuracy.

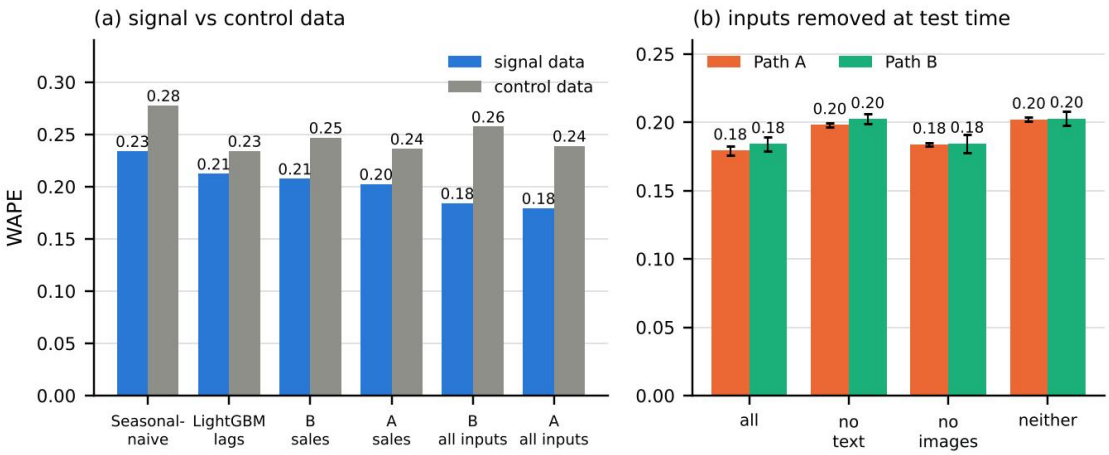


Figure 4.6: Control experiment and robustness: (a) key approaches on signal and control data; (b) inputs removed at forecast time

Figures 4.7 and 4.8 show the running application (quick setting: 12 epochs, one seed, so values differ slightly from Table 4.4; the interface warns that one seed may be noise).



Figure 4.7: Comparison tab: verdict, error chart and table

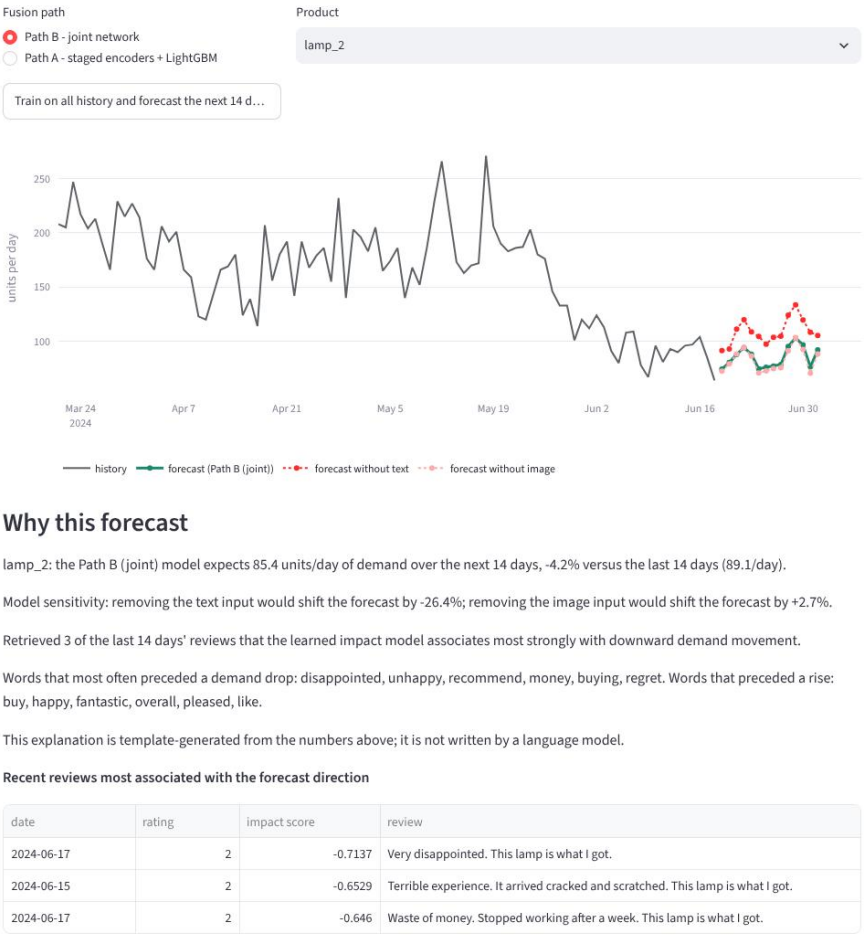


Figure 4.8: Forecast and explanation tab: 14-day forecast, explanation and retrieved reviews

4.3.1 Sample Run on the Team's Own Machine

In order to prove that the program can work independently from the author's experiment environment, the authors executed the program on their computer and made the following screenshots without any changes or crop (each figure is a full screen, including a sidebar with parameters used). The run involves the control data source (reviews and images do not have any information about the demand), 9 products, 1000 days of history, and training effort "Thorough, 3 seeds". The control data source was selected on purpose because it allows testing whether the system creates a profit from input data, which do not have anything. This statement is also made in the note of the application under the robustness table.

Step 1 - hold-out error (Figure 4.9). After "Load data and run comparison" on the Comparison tab, each approach is ranked according to their WAPE performance during the hold-out period of the final 120 days. The colors categorize each family of algorithms: light grey for the baselines, dark blue for LightGBM on lags, light green for the joint network (path B) and orange for the staged model (Path A); the thin black lines represent the range among the different seeds. The model performing lowest is LightGBM on lags (0.209); equally performing is the joint network based on either sales (0.210) or sales and images (0.211); all the staged approaches fall between 0.215 and 0.216 while the joint network with text (0.218) or all the inputs (0.221) are the worst-performing learned models.

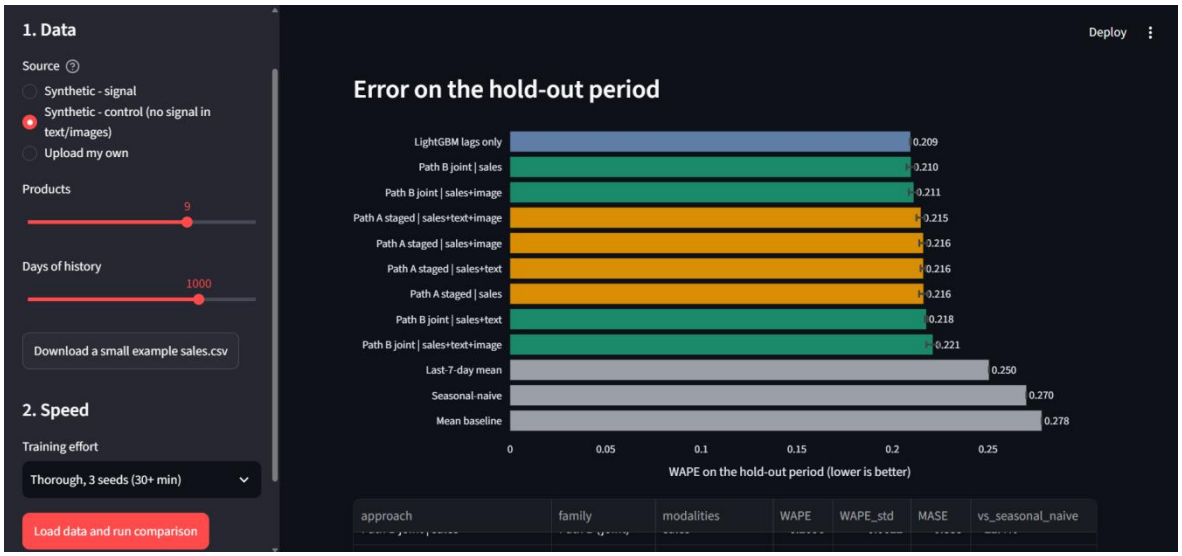


Figure 4.9: Sample run, control data (9 products, 1000 days): error on the hold-out period for all approaches, with seed whiskers

Step 2 - error by forecast distance (Figures 4.10 and 4.11). The "Error by forecast distance" chart shows the relationship between WAPE and forecast days ahead. Each method can be enabled or disabled in the selection box, Figures 4.10 presents all twelve methods in the list while Figure 4.11 compares only two full-input methods (Path A Staged and Path B Joint), where both of the curves have a constant increase along forecast distance – an understandable phenomenon since more distant days are more difficult to predict. The joint network curve starts from a little bit lower point (around 0.165 on day 1 compared to 0.172 of the staged model) but reaches a much higher level (around 0.26 on day 14 vs. 0.247 of the other one), and crosses each other at around day 3 to 4.

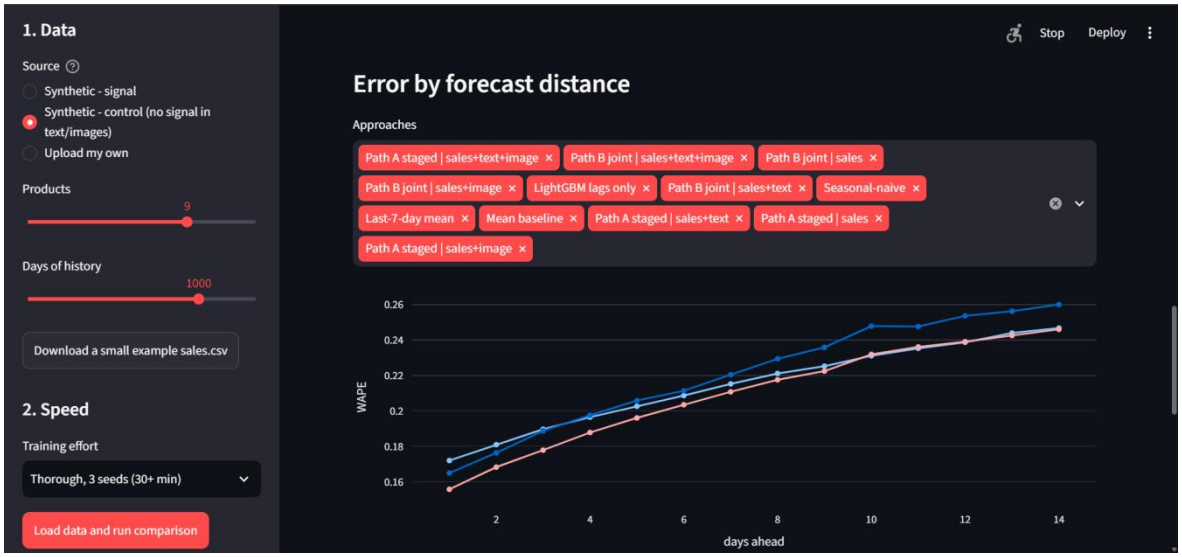


Figure 4.10: Sample run: error by forecast distance with all approaches selected



Figure 4.11: Sample run: error by forecast distance for the two full-input models, Path A (staged) and Path B (joint)

Step 3 - what if an input goes missing (Figure 4.12). In the robustness table, the models trained on all the inputs are re-scored by concealing text, image, or both during the forecast. In this case, the numbers remain constant: the staged model retains its 0.215 across all the four lines, while the joint network changes only to 0.222 when both inputs are concealed, coming from 0.221. This is the desired and honest result for the control input – the concealment of the input that does not contain any information cannot harm – and it differs from the signal input, where the removal of text results in the increase of the error rate from 0.179 to 0.198 in Path A (Figure 4.6b). The footnote added under the table in the application restates that the benefit from text or image in the "signal" mode comes from the design of the generator.



Figure 4.12: Sample run: robustness table with inputs hidden at forecast time

Step 4 - forecast for one product (Figures 4.13 to 4.15). The Forecast tab trains the selected path on all history and forecasts the next 14 days starting from the last day. The user selects the fusion path (either Path B joint network or Path A staged encoders + LightGBM) and the product through a drop-down list. Figure 4.13 shows Path B for kettle_1, Figure 4.14 shows the product list containing the nine products (from kettle_1 to mixer_9), and Figure 4.15 shows Path A for backpack_3. In all graphs the grey curve is recent history, the green curve – forecast, and two dotted curves are the same forecast, calculated twice without text and without images respectively; the colored dots represent 14 days in the forecast. On the control data the three forecast curves completely coincide, which means visually the flat robustness table: models do not take review and image inputs into account. The forecast curves are non-flat: they go up and down in a cyclic way (there are two maxima on the graph for backpack_3 one week apart), reflecting the weekday effect learned from the sales history.

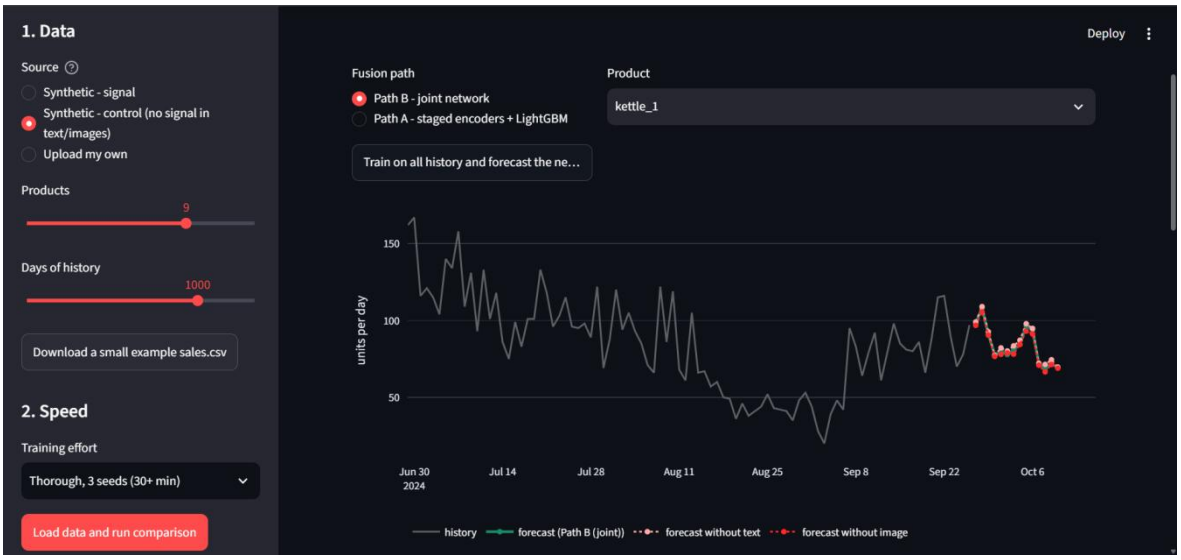


Figure 4.13: Sample run: 14-day forecast for kettle_1 with Path B (joint network)

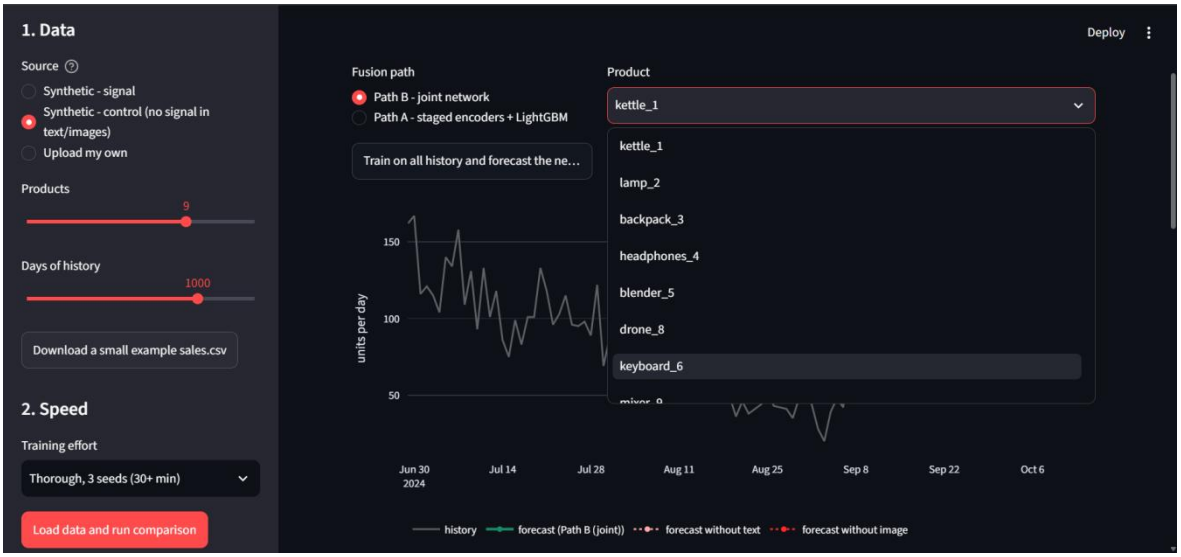


Figure 4.14: Sample run: choosing the product from the drop-down list

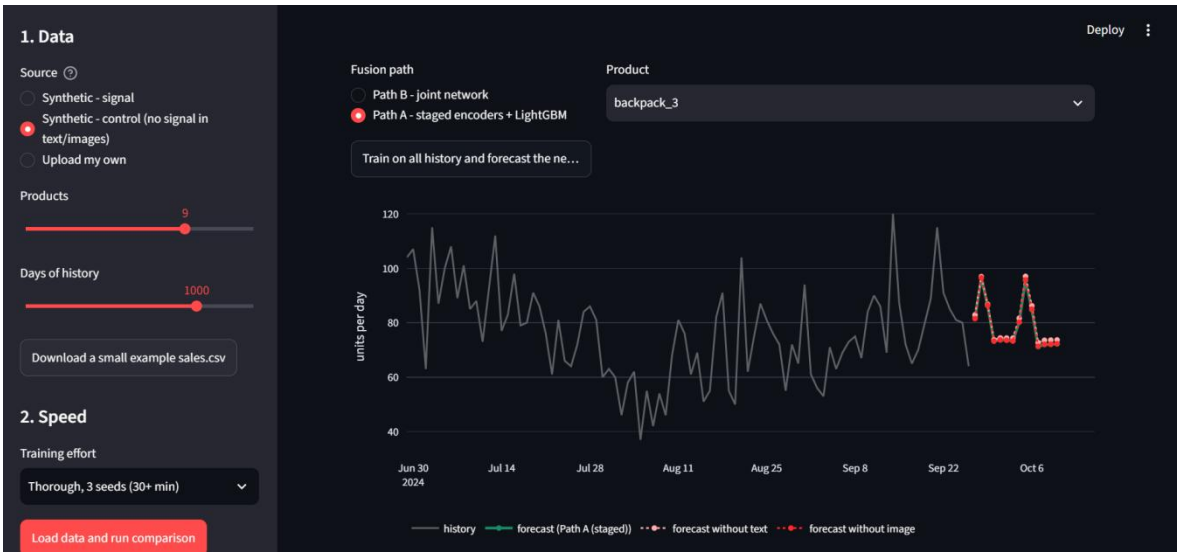


Figure 4.15: Sample run: 14-day forecast for backpack_3 with Path A (staged encoders + LightGBM)

Table 4.5: Numbers shown in the sample run (control data, 9 products, 1000 days, hold-out WAPE)

Approach	WAPE	Remark
LightGBM on lags only	0.209	Best overall
Path B joint, sales / sales + image	0.210 / 0.211	Level with LightGBM
Path A staged, all inputs	0.215	Unchanged when inputs are hidden
Path A staged, sales + image / sales + text / sales	0.216 / 0.216 / 0.216	No gain from extra inputs
Path B joint, sales + text / all inputs	0.218 / 0.221	Slightly worse with noise inputs
Last-7-day mean / Seasonal-naive / Mean	0.250 / 0.270 / 0.278	All beaten by the learned models

With reference to the sample run and Table 4.4: on the signal data, the text and image combination is the best model for Model 1 by 11% compared to the best sales-only model, whereas on the control data, the same code provides no improvement and the simplest robust model (LightGBM on lags) is as effective as any neural network variant. The two findings are consistent with a system that uses an input only when the input contains information. The values are different from the control values in Section 4.3 because this run employs more products and more history, a different training effort value and its own seeds; the order of the approaches, not the third digit, is the finding. The differences between the learned models in Table 4.5 are small (0.209 to 0.221) and the seed whiskers in Figure 4.9 are about the same size, so the run does not indicate that any of them is better.

4.4 Quality Assurance

Quality was ensured through (i) the 31-test set, executed in two package environments; (ii) leakage detection through both construction and perturbation tests; (iii) fair evaluation: a no-signal control, three random seeds, a decision not to crown a victor under noise and given limitations; and (iv) informative messages for all conceivable data problems. Issues identified and resolved: the initial leakage test erroneously detected a signal since the scaling and vocabularies are trained on training days (the test has been adjusted to remove only data after the origin); a "+" in the model name would cause an error in Keras; the repetitive warnings from parsing dates were disabled; and on Windows, the installation of requirements was not possible as TensorFlow had long header paths over the 260-character limit, which could be solved by using a shorter path for the virtual environment.

Chapter 5 Standards Adopted

5.1 Design Standards

The requirements adhere to IEEE Std 830 and ISO/IEC/IEEE 29148 [19, 20]; the design follows IEEE Std 1016 [22] (layer architecture, table of modules, data structure); the non-functional requirements follow the ISO/IEC 25010 quality characteristics [23] (suitability, efficiency, interoperability, usability, reliability, security, maintainability, portability).

5.2 Coding Standards

This program meets the guidelines laid out in PEP 8 and PEP 257 [25],[26]. Descriptive variable names, single-functionality functions, type annotations, dataclass to represent data, use of no magic numbers, only one configuration object, seeding the random generator and ValueError exceptions with helpful messages; linting not done yet.

5.3 Testing Standards

The tests are based on the levels and methods of ISO/IEC/IEEE 29119 [24] (equivalence partitioning, boundary value analysis, negative testing) and the IDCR method of Table 4.3; the model results rely on chronological hold-out validation [1], [14], leakage test [13], repeated runs and control experiment.

5.4 Cost Analysis

The software used is all freely available under open-source licenses (Python, TensorFlow/Keras/Streamlit: permissive licenses; scikit-learn, pandas, NumPy: BSD; LightGBM, Plotly, pytest: MIT), and thus have no license fee costs associated. No graphics processing unit nor any cloud services were used; all tests were

performed using only a two-core CPU which took around 5-9 minutes And the fast application setting takes just a couple of minutes, hence making the running costs be those of energy consumed by the ordinary computer. The design was estimated to take about two weeks per developer. Maintenance is inexpensive since the code is modular and tested. Business forecasting systems are always subscriptions, where one is expected to send data to the system; this system however has its data stored locally and is modifiable, though it is still a research project (12 products per run, no live data connection, no real-data validation).

Chapter 6 Conclusion and Future Scope

6.1 Conclusion

The project developed a multimodal demand-forecasting model that includes a GRU and 1D-CNN for the sales, TF-IDF with LSA for the reviews, a CNN for images, and a retrieval-based explanation, and compared staged and joint fusion under identical and non-leaky conditions on synthetic data. Review text decreased the error by about 0.02 WAPE (about 11 %) for both designs, with the best design achieving 0.179 against 0.234 for seasonal-naive. Images did not help much with the very small from-scratch CNN. Joint and staged fusions did not differ significantly in terms of accuracy. Nothing was gained in the control experiment using uninformative inputs, while the jointly-trained network degraded by about 0.01 WAPE because of learning the noise, which makes staged fusion more preferable when there is uncertainty in the usefulness of an input. Both designs degrade gracefully to sales-only accuracy in case of missing inputs. Additionally, the system satisfies all of its engineering requirements: partial uploads work and explainable, evaluation is non-leaky and validated, and it runs on a regular CPU.

Limitations. They are gains seen through the lens of synthetic data wherein the reviews and images precede sales, demonstrating the ability to make use of an existing signal, but not the amount gained in real stores; the network for images is quite simple and is trained using 32 by 32 images; text uses latent semantic analysis and not pre-trained language modeling; the explanations are based on retrieval and a template; and there are three seeds used and one data set.

6.2 Future Scope

(1) Validation on actual retail data with actual reviews and images. (2) Using pretrained image and sentence encoders in the same staged-versus-joint setup. (3) Adding price, promotion, out-of-stock and holiday covariates. (4) Providing probabilistic predictions (quantiles). (5) Removing the 12 product cap and forecasting hierarchically. (6) Creating fluent explanations using a language model restricted to the evidence retrieved and measuring faithfulness. (7) Repeating the experiment on many synthetic data sets with significance testing and equal hyperparameter tuning for both paths. (8) Containerisation, monitoring of drift and linting/continuous integration.

References

[1] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.

[2] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *International Journal of Forecasting*, vol. 22, no. 4, 2006.

[3] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, 1997.

[4] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. EMNLP*, 2014.

[5] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, 1998.

[6] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[7] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, 1988.

[8] S. Deerwester, S. T. Dumais, G. W. Furnas, T. K. Landauer and R. Harshman, "Indexing by latent semantic analysis," *Journal of the American Society for Information Science*, vol. 41, no. 6, 1990.

[9] T. Baltrusaitis, C. Ahuja and L.-P. Morency, "Multimodal machine learning: A survey and taxonomy," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 2, 2019.

[10] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, 1992.

[11] N. Neverova, C. Wolf, G. Taylor and F. Nebout, "ModDrop: Adaptive multi-modal gesture recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 8, 2016.

[12] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[13] S. Kaufman, S. Rosset, C. Perlich and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, 2012.

[14] C. Bergmeir and J. M. Benitez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, 2012.

[15] S. Makridakis, E. Spiliotis and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," *International Journal of Forecasting*, vol. 38, no. 4, 2022.

[16] D. Salinas, V. Flunkert, J. Gasthaus and T. Januschowski, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks," *International Journal of Forecasting*, vol. 36, no. 3, 2020.

[17] B. Lim, S. O. Arik, N. Loeff and T. Pfister, "Temporal Fusion Transformers for interpretable multi-horizon time series forecasting," *International Journal of Forecasting*, vol. 37, no. 4, 2021.

[18] V. Ekambaram et al., "Attention based multi-modal new product sales time-series forecasting," in *Proc. ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2020.

[19] IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE, 1998.

[20] ISO/IEC/IEEE 29148:2018, *Systems and software engineering - Life cycle processes - Requirements engineering*, 2018.

[21] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[22] IEEE Std 1016-2009, *IEEE Standard for Information Technology - Systems Design - Software Design Descriptions*, IEEE, 2009.

[23] ISO/IEC 25010:2011, *Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models*, 2011.

[24] ISO/IEC/IEEE 29119 (Parts 1 and 3), *Software and systems engineering - Software testing*, 2013.

[25] G. van Rossum, B. Warsaw and A. Coghlan, "PEP 8 - Style guide for Python code," Python Enhancement Proposals, 2001.

[26] D. Goodger and G. van Rossum, "PEP 257 - Docstring conventions," Python Enhancement Proposals, 2001.

Individual Contribution

INDIVIDUAL CONTRIBUTION REPORT: MULTIMODAL DEMAND FORECASTING

Soham Chatterjee
2328128

Abstract:

The proposed approach forecasts 14-day demands for the products based on their sales history, review texts, and product images. It involves the combination of GRU and 1D-CNN sales encoder, LSA-based review encoder, and image CNN and evaluates two fusion strategies – staged (involving separately trained encoders feeding a LightGBM stacker) and joint (a single end-to-end trained network) on the same, non-leaky data. Text or images could be absent in any case, and the system still works. The predictions are supported with a retrieval of the latest reviews. With the use of synthetic data, the review text reduces WAPE by 0.02 and the best-performing model achieves 0.179 against 0.234 of the seasonal-naive baseline; the two fusion architectures perform equally well, while the joint architecture underperforms the control one by 0.01 WAPE where the reviews and images are not informative at all.

Individual contribution and findings:

Responsibility: models (hff/nets.py, hff/fusion.py, hff/forecast.py, hff/explain.py) and tests/test_models.py.

- **Network components (hff/nets.py).** •Sales encoder (GRU with 32 units plus 1D convolutions with average and maximum pooling, resulting in 32 dimensional embedding); Text encoder (1D Convolution plus 16 unit GRU); Image CNN (Three times 3×3 Convolutions with pooling). The same prediction head is trained to predict the difference from the previous week's value for all 14 days. A ModalityGate layers multiply embeddings by their existence flag and dropout whole modalities with probability 0.25.
- **The two fusion paths (hff/fusion.py).** •Path A trains the encoders individually on the initial 60% of training sources, freezes them, and trains the LightGBM stacker model (250 trees, L1 loss) on the remaining sources with a 14-day embargo period. Path B comprises a single neural network with a single loss function trained end-to-end. There's also the LightGBM lag-based baseline. Parameter counts are 14,510 (sales only), 18,430 (sales + text) and 23,790 (all features).
- **Final forecast (hff/forecast.py).** Trains on the selected path through all history, makes forecast for the following 14 days, and then does re-forecast by hiding either text or image inputs to see their impact.
- **Explanation module (hff/explain.py).** •A ridge regression of TF-IDF vectors on the standardized change in sales for the upcoming 7 days; fetches top 3 relevant reviews out of the last 14 days' review set, identifies the words before increases/decreases, and completes the template form. It is deliberately template-based and not a language model.
- **Tests (tests/test_models.py).** •Joint model with text and image inputs masked (the output remains finite and varies, indicating the use of the text input); the benchmark using full and sales-only data; forecast and explanation of both paths, including the sales-only path.

Findings. There is no statistically significant difference between staged and joint fusion if reviews have information (WAPE 0.179 vs 0.184, within seed range). Joint model performs around 0.01 WAPE worse than the sales-only model in case of uninformative reviews and pictures, whereas staged fusion is not affected. Masking text at the time of prediction increases WAPE by around 0.02 for both models, and masking both inputs results in the sales-only WAPE.

Individual contribution to the project report preparation: Chapter 2 (Basic concepts and literature review, Table 2.1); Section 3.3 (System design and architecture); Section 4.1 (Methodology: encoders and the two fusion approaches, Figures 4.3 and 4.4).

Individual contribution for the project presentation and demonstration:

Discussion on the architecture of the model and the staged vs. joint approach has been done, and the forecast and explanation tabs have been illustrated, namely forecast, forecast without text/images, and retrieved reviews.

Full Signature of Supervisor: _____ Full signature of the student: _____

INDIVIDUAL CONTRIBUTION REPORT: MULTIMODAL DEMAND FORECASTING

Avrrodeep Banerjee
2328158

Abstract:

Project predicts 14-days ahead product demand from the past sales records, reviews text, and images. It utilizes GRU sales encoder coupled with 1D-CNN architecture, review encoder based on Latent Semantic Analysis and image convolutional neural network, and explores two different fusion methods applied to the same leak-free input data: staged and joint fusion (in case of joint fusion – one network is trained end-to-end). Either text or images can be missing but the system still produces predictions. Predictions are justified by identifying relevant reviews. On the synthetic data set, review text reduces WAPE by 0.02 and the best result equals 0.179 versus 0.234 for the seasonal naïve algorithm; both fusion architectures are statistically comparable, while joint architecture underperforms by 0.01 WAPE compared to the control data set where both text and images are informative-less.

Individual contribution and findings:

Responsibility: data_processing (hff/ingest.py, hff/config.py) and tests/test_ingest.py.

- **Ingestion and validation (hff/ingest.py, about 450 lines).** •Parses the user's sales, reviews and images files and creates one package of validated data. It detects the column names from the alias (date; sales, units/quantity; product, store/item; review; rating), detects the dates, matches the product keys in the three data sources with an alert when there is no name available.
- **Missing and partial data.** The sales with up to 7 days gap are interpolated while the rest are backfilled using the median of products and marked as such so that the model will know the difference between actual and filled days. When there is no sales data but only reviews then the number of reviews per day becomes the demand figure and this fact is noted. Each input receives a flag indicating its status with a description: reviews are disabled below 50 or 2% and images older than 21 days**.**
- **Image and file handling.** Reads an image ZIP either via file names like kettle_2024-03-05.png or via manifest.csv, ignores corrupt images with error reporting, resizes to 32 x 32 pixels. Converts the generated synthetic images into the same fgle place for tormat, so that synthetic and uploaded images share the same pipeline code..in
- **Configuration (hff/config.py).** Heronizon (14 days), window (56), hold-out (120 days), size of image, dropout (0.25) and training parameters, to make every experiment replicable.
- **Tests (tests/test_ingest.py, about 120 lines).** •Ingestion matrix for sales only, sales and review, sales and image, all three, partial data, messy column names, 10% missing rows, corrupt images, and inadequate history.

Findings. The ingestion layer worked fine for all combinations of both existing and non-existing inputs in the test matrix and would produce results that would be either useful or an explainable output for sure. Turning off a modality instead of learning on it would safeguard models from being trained on very limited reviews/images..

Individual contribution to the project report preparation:

Abstract; Introduction (Chapter 1); Project Analysis and requirement specification (section 3.2); Testing and verification plan, Test cases T1-T9 (section 4.2); inputs given to the models (Table 4.1).

Individual contribution for the project presentation and demonstration:

Stated the problem along with the three inputs and showed how to upload data with missing modality (sales alone followed by sales along with incomplete reviews).

Full Signature of Supervisor: _____ Full signature of the student: _____

INDIVIDUAL CONTRIBUTION REPORT: MULTIMODAL DEMAND FORECASTING

Debarshi Karmakar
2328164

Abstract:

The project predicts the 14-day demand of the products using sales history, review texts and product images. It uses a combination of GRU with 1D-CNN sales encoder, LSA review encoder and an image CNN, and compares two architectures for combining them based on the same and unbiased data: staged fusion (where the encoders are separately trained and their outputs fed into a LightGBM stacker) and joint fusion (where one network is trained end-to-end). Either the review text or the images can be absent, but the system still works. The explanations of predictions are provided by extracting the most recent reviews. In the case of synthetic data, review text reduces WAPE by 0.02 and the best model achieves 0.179 compared to 0.234 of the seasonal-naive baseline; the two fusion architectures are not statistically different, and the joint network underperforms by 0.01 WAPE on a control dataset with no predictive power of reviews and images.

Individual contribution and findings:

Responsibility: app.py and finding the dataset (hff/synth.py, sample_data/, tests/test_synth.py, tests/test_app.py).

- **Finding and choosing the dataset.** Couldn't find any publicly available dataset with matching sales, reviews text, and product images for the same products and dates. Hence decided to create a synthetic data generator where the architecture is known, so that the claims can be verified, and created two modes - Signal & Control.
- **Synthetic generator (hff/synth.py).** •A hidden product-quality process (AR(1), coefficient 0.97, with defect episodes) influences sales via base level, trend, weekday, year and promotion factors, quality factor with six-day lag, and Poisson noise. The reviews come in at 0.05 times the number of sales with some template text and 1–5 quality score following the quality; one 32×32 image with scratches reflecting lost quality is generated every 7 days. The default dataset has 6 products × 900 days, 25,353 reviews and 771 images. In the control regime, the three quality processes are independent, hence reviews/images contain no information.
- **Sample files (sample_data/).** Three pre-made examples for uploading (full, sales only, reviews only) created by the generator, along with a small example file for sales downloadable via the app.
- **Streamlit application (app.py, about 230 lines).** •Sidebar with data source, product and day sliders, and training effort; three tabs (Data check, Comparison, Forecast and explanation); Plotly graphs for error comparison, error per horizon, and forecast; download in CSV format; clear error messages. An environment variable makes automated testing possible with less epochs.
- **Tests and screenshots.**tests/test_synth.py (generator is reproducible; signal data correlates with its leading feature, control data doesn't; reviews and images provided); tests/test_app.py (guidance on start up, end-to-end synthesis run, sales upload only, and error instead of crash in case of no upload); Playwright screenshots included in the paper/report.

Findings. Quality #1 in the signals dataset is correlated with subsequent sales above 0.15 and with below 0.1 in the controls dataset; that's why the control experiment was so useful. Time of application execution varies from 2 to 4 minutes at fast option setting.

Individual contribution to the project report preparation:

Chapter 3.1 (Project planning); Data generation process of Chapter 4.1 (Figures 4.1); Chapters 4.3 and 4.3.1 (Application Screenshots and the Sample Run, Figures 4.7 to 4.15); Chapter 5 (Standards Adopted and Cost analysis); Figures creation script for report and paper.

Individual contribution for the project presentation and demonstration:

Explained how the dataset was selected and created and then ran the live application all the way through from synthetic data to the comparison and the explanation of the forecast.

Full Signature of Supervisor: _____ Full signature of the student: _____

INDIVIDUAL CONTRIBUTION REPORT: MULTIMODAL DEMAND FORECASTING

Supratik Maity
2328226

Abstract:

This project makes the forecast for 14 days of product demand based on sales history, review text and product images using GRU with 1D-CNN sales encoder, LSA-based review encoder and an image CNN, comparing both approaches for fusion in the same conditions of the data used (without leaks): staged approach (separate models trained and then passed into LightGBM stacker) and joint approach (model trained together in one network). Both the review text and images may be absent and the approach is still applicable. Explanation of forecasts is made through retrieval of recent reviews. With synthetic dataset the use of review text decreases WAPE by around 0.02 and the best model gives WAPE of 0.179 versus 0.234 achieved with seasonality-naive baseline; two ways of fusion are statistically equivalent, with the joint network giving additional 0.01 WAPE increase compared to the baseline with control dataset that contains no useful information in review and images**.**

Individual contribution and findings:

Responsibility: features and evaluation (hff/windows.py, hff/benchmark.py, scripts/benchmark.py) and tests/test_windows.py.

- **Features and leak-free windows (hff/windows.py). •**Constructs each training instance using data from until forecasted date alone: a 56-day sales history with 6 sources of channel data, 10 manually-engineered features (latest value, lags at 7th, 14th and 28th days, moving averages, SD, trend, observation rate), same weekday values, and calendar dummy variables. Scaling is done per item in training. Feature set for review data uses TF-IDF and latent semantic analysis on review data alone in the training period (16 dimensions along with rating, log and occurrence). Image look-up is the latest available picture in 21 days before.
- **Chronological split.** Past 120 days hold-out (642 forecasting origins), where training origins stopped 14 days prior to ensure that the hold-out is not overlapping the training origin.
- **Evaluation (hff/benchmark.py).** WAPE, MASE, and WAPE by forecast horizon; four baseline models (mean, last 7 days mean, seasonal naive, and LightGBM with lagged data); both approaches for all subsets of modalities (ablation study); multiple seeds; a robustness test using hidden inputs during forecasting; and a decision rule that considers differences to be significant only if more than double the seed standard deviation.
- **Command-line benchmark (scripts/benchmark.py).** Performs the same comparison but without the user interface and generates the tables for the signal and control results used in the report and the paper (3 seeds in 8.6 minutes with 2 CPU cores).
- **Tests (tests/test_windows.py).** Overwrite all data following a forecast origin and check that the inputs at that forecast origin do not change, and a check that scaling is done using the training period only.

Findings. Best model (Path A, all inputs) achieves WAPE of 0.179 and MASE of 0.869, 23.3% better than seasonal naive (0.234). Text review is responsible for around 0.02 of the improvement; images contribute very little (around 0.005 for Path A, nothing measurable for Path B). The control run demonstrated that the improvements vanish without informative inputs..

Individual contribution to the project report preparation:

4.3 (Result analysis: Table 4.4, Figures 4.5 & 4.6); 4.4 (Quality Assurance); 6 (Conclusion & Future Scope); Table 4.2 (Configuration).

Individual contribution for the project presentation and demonstration:

Explained the evaluation design (split, gap, and leakage tests and measures) and the findings, which include the control test and the robustness test**.**

Full Signature of Supervisor: _____ Full signature of the student: _____

Plagiarism Report

Attach here the plagiarism (similarity) report generated for this project report by the institute's checking tool, with the similarity index visible. This page is left for the report because it can only be produced from the final document.